

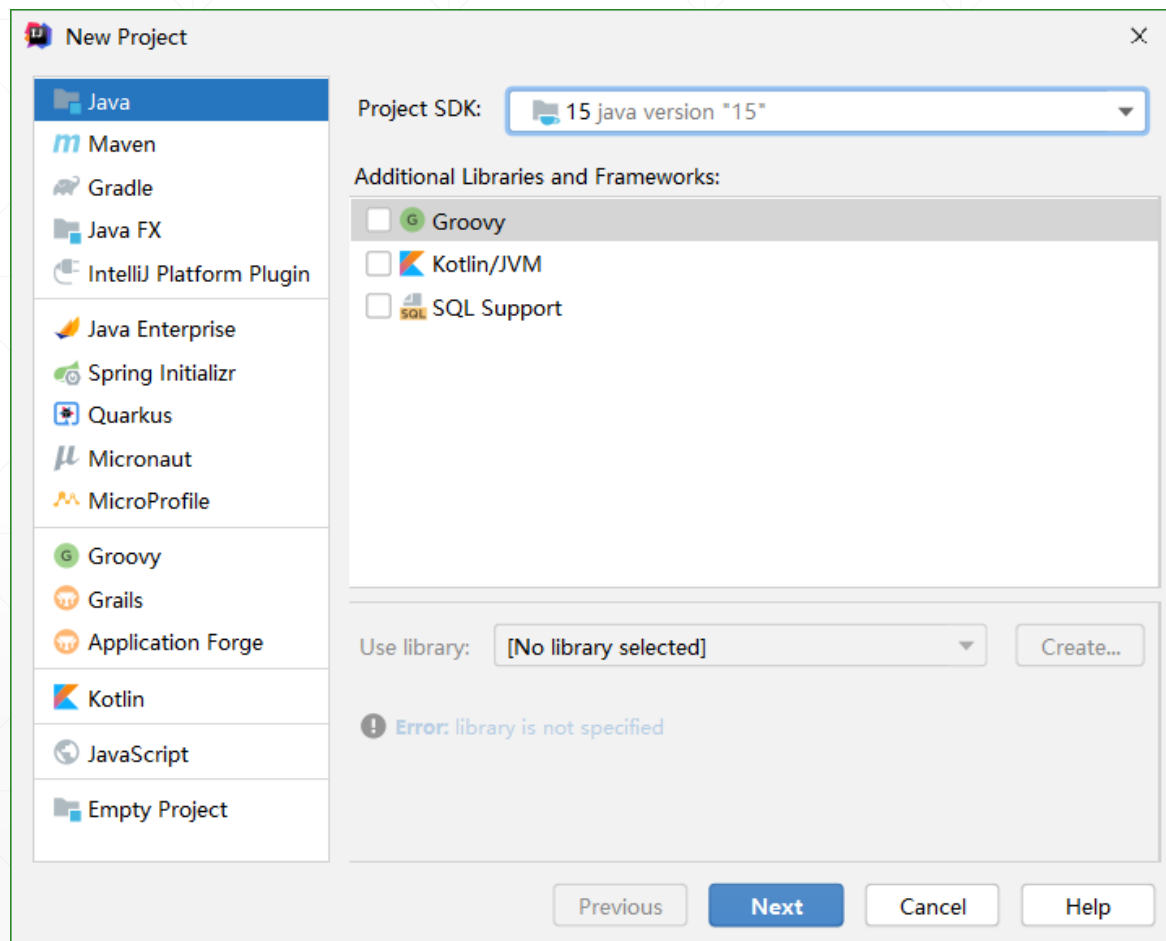


开发简单的Java应用程序

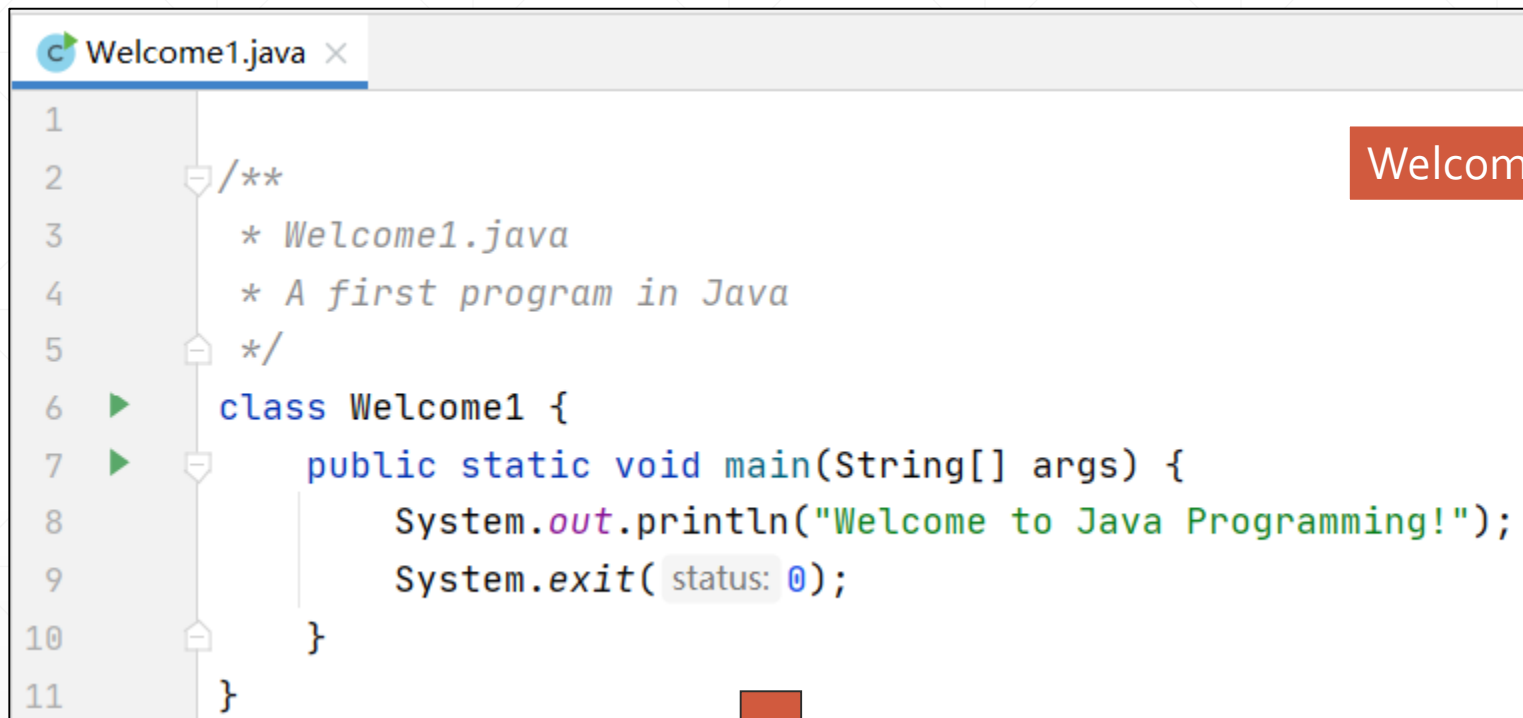
北京理工大学计算机学院
金旭亮

Java可以开发的项目类型

- Java可以开发多种类型的项目，每种类型的项目都有特定的应用场景。
- Java Application (Java应用程序) 是最简单的一种，是入门的最佳选择。
- 在后继的学习中，会接触到其它的项目类型。

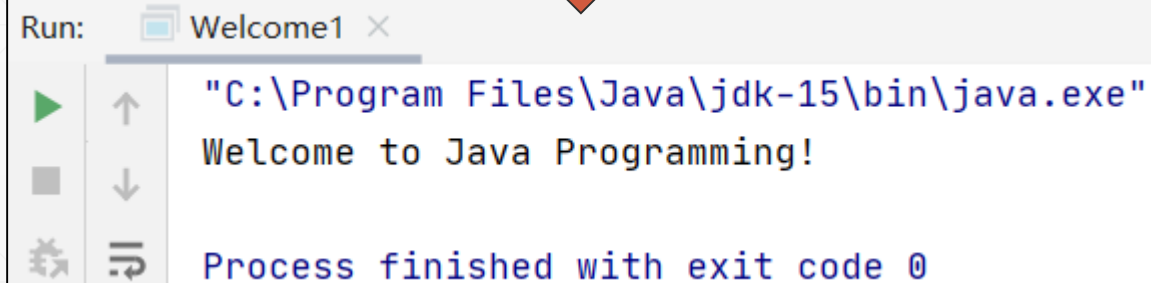


第一个Java应用程序



```
1  /**
2   * Welcome1.java
3   * A first program in Java
4   */
5
6  class Welcome1 {
7      public static void main(String[] args) {
8          System.out.println("Welcome to Java Programming!");
9          System.exit(status: 0);
10     }
11 }
```

Welcome1.java



```
Run: Welcome1 x
"C:\Program Files\Java\jdk-15\bin\java.exe"
Welcome to Java Programming!
Process finished with exit code 0
```

Java中的注释

- 单行注释: `//`
 - 注释用于说明程序开发者的意图，不是语法说明！
 - 注释可以提高程序可读性
 - 在开发时为代码添加适当的注释是一个良好习惯
- 多行注释: `/* ... */`

```
/* 这是一个多行的注释  
   可以分布于多行 */
```

代码中的“空白符”



代码中添加适当的空白符，能提升程序的可读性



空行、空格和tab被称为“**空白符（white-space characters）**”，会被编译器所忽略



程序中的空行，一般用于给功能代码分组，给代码添加适当数量的空行，是一个良好的写代码的习惯，能有效地提升代码的可阅读性。



依据代码的结构，使用Tab或空格给语句添加相应的缩进，在运算符与操作数之间添加空格，都是好习惯，IntelliJ提供了格式化代码的功能（Ctrl+Alt+L）将这些功能“自动化”了。

一种“常见的误解”

- 猜想:

在程序中写太多的注释，会使程序变大，运行变慢……

- 回答:

不会!



扩充学习——Java Document



为了减轻Java程序员人工编写文档的工作量，实现从代码中自动地抽取注释的功能，Java中定义了一种标准的注释方式，内置了一些标准的以“@”开头的关键字：

```
/**  
 * @author JinXuLiang  
 * @version 1.0  
 */
```



使用 Javadoc 程序可以从代码中抽取上述注释内容，并创建出HTML格式的程序文档。JDK文档就是使用这种方法生成的。

Java中的语句

- Java语句以分号;结束
- 一个单独的语句可以被分为多行，可以多敲回车避免一行语句过长，难于阅读。



不能在一个标识符中间或字符串内部断开一个语句。

Welcome1示例逐句解析：定义类

```
public class Welcome1 {
```

- 开始定义类Welcome1
 - 每一个Java程序至少有一个用户自定义的类
 - 关键字 (**KeyWord**)：有特殊的含义、专供Java本身使用的单词，Java编译器能识别它们。关键字不能用于定义变量或类名。
- 关键字class后紧跟的是类的名字
 - Java类名规范：每个单词的首字母大写，例如：**SampleClassName**

Java中的标识符

- 类名和变量名都是Java “标识符 (identifier)”。
- Java中的标识符可以由字母、数字、下划线（即“_”）和 \$ 构成，不允许以数字开头，也不允许有空格，比如：Welcome1、\$value、_value、button7是有效的标识符，而7button 无效。
- 尽量不要用“\$”打头，在Java中，“\$”一般用于标识内部类生成的.class文件。
- 请遵循通用的命名规范，i、j、k、x、y之类含义模糊的标识符名字，要尽量少用。



Java标识符是大小写敏感的，比如a1和A1是不同的标识符。

Java源文件名必须与公有类名一致

类源文件名 = 公有类名 + .java



1. 如果一个类被声明为public，则它本身所在的源文件名也必须与类名相同，连大小写都不能错！
2. 这并非说一个Java源文件中只能写一个类，你完全可以在一个.java文件中写多个类，但其中只能有一个类是“公有（public）”的，并且Java要求文件名也要与之一致。

置疑:



- 一个Java类文件中真的只能有一个公有类吗?
- 请使用IntelliJ或javac检测一下以下代码,有错吗?

```
public class Test {  
  
    public static void main(String[] args) {  
  
    }  
  
    public class InnerClass{  
  
    }  
  
}
```

请依据你的试验结果调整前页的描述,使其更为准确。
在后面的课程中我们还将学习接口(interface), 这里关于公有类的结论是否同样可用于接口?

main方法

- Java中的方法用于完成某个任务并返回结果信息，一个Java应用程序可以包容多个类，每个类又可以包含一个或多个方法。
- Java应用程序（Java Application）从 main 方法开始执行

```
public static void main( String[] args )
```

- main 方法必须严格象上面那样声明
- void 表明 main 方法不返回任何结果
- 包容有main方法的类称为“**主类 (Main Class)**”。



动手试验:

把main()方法的返回值由 void 改为 int，程序能编译通过吗？能运行吗？

Java Application中的输出语句

```
System.out.println( "Welcome to Java Programming!" );
```

- System.out.println打印一个字符串
 - Java中的字符串由双引号包围（**注意不是中文的双引号**）
 - 字符串中的空白符会被编译器保留下来。
- 整行代码是一条“**语句（statement）**”，程序语句命令计算机执行特定的指令，本句代码在执行时，会在命令提示符窗口输出一个字符串——Welcome to Java Programming!
- System是“**类（class）**”，out是这个类的一个“**字段（field）**”，“System.out”引用Java所提供的“标准输出对象”，println是这一对象所拥有的“**方法（method）**”，其功能是——在命令提示符窗口中输出内容。

试一试，手工编译和运行示例程序



- 打开一个命令提示符窗口，进入源文件所在的文件夹，输入：

```
javac Welcome1.java
```

- 如果没有错误，将生成 Welcome1.class，此文件中保存了Java编译器生成的字节码
- 当Java程序运行时，Java 解释器（interpreter）可以解释这些字节码
- 要执行Java字节码，输入：

```
java Welcome1 （扩展名 “.class” 必须省略）
```

- 解释器将装入类 Welcome1的 .class 文件，然后调用方法main()启动执行

应用格式控制符分行显示

System.out.print 和 System.out.println可接收一些特殊的格式控制符（又称为“**转义字符**”）。转义控制符以反斜杠引导(\)，具有特定的含义。比如“\n”代表换行符，通知命令提示符窗口将光标移到下一行开头。

转义字符	字符	转义字符	字符
\\	反斜杠	\f	换页
\b	退格	\t	横向跳格
\r	回车	\n	换行
\"	双引号	\'	单引号

阶段小结:



从示例Welcome1.java中，我们学到了...

- Java程序的基本组成单位是类
- 类包容方法
- main()方法是程序入口点
- 要编程时要注意养成良好的编程习惯

示例二：在一个消息框中显示文本

```
// Printing multiple lines in a dialog box
import javax.swing.JOptionPane; // import class JOptionPane

public class Welcome2 {
    public static void main(String args[]) {

        JOptionPane.showMessageDialog(null, "Welcome\nto\nJava\nProgramming!");

        System.exit(0); // terminate the program
    }
}
```

Welcome2.java



- Java应用程序可以使用窗口或对话框，相应的开发框架是AWT/Swing或JavaFX
- JOptionPane类允许我们使用消息框来显示一些信息，它是Swing提供的组件。

包 (package)

- JOptionPane 在包 javax.swing 中，此包中有一些可用于 “**图形界面 GUI (Graphical User Interfaces) 设计**” 的组件。
- 在Java中，相关的类被放在一起，称为 “**包 (package)**”
- JDK预置的所有的包构成 “**Java标准类库**” 或 “**Java应用程序编程接口 (Java API: Java Application Programming Interface)**”。



开发有可视化界面（比如窗体）Java应用的“图形界面（称为GUI）”开发框架有好几个，Swing是其中一种，比较古老，但一直内置于JDK中，始终可用。

另一种功能更为强大也更为现代的GUI框架叫做JavaFX，我们在后面的课程中会介绍它。

Java 包的种类

- **核心包 (Core packages)**：以 “java” 开头，由JDK所直接提供
- **扩展包 (Extension packages)**：以 “javax” 开头
- **第三方包**：通常以 “反写” 的公司网址开头。不过这仅是一种约定和惯例，并非强制性的。比如某包名为：com.example，它实际上代表一家公司，其网址通常为：<http://www.example.com>。

包的导入

使用某类之前，如果这个类所在的包不是使用它的代码所在的包，那么，你需要使用 import 语句来导入它所在的包：

```
import javax.swing.JOptionPane;
```

- import 语句供编译器用来识别和定位Java程序中用到的类。
- 上述语句告诉编译器在 javax.swing 包中装入类 JOptionPane 的定义，这样在源程序代码中就可以使用它了。

使用消息框

```
JOptionPane.showMessageDialog(  
    null, "Welcome\nto\nJava\nProgramming!" );
```

- 调用类JOptionPane的showMessageDialog 方法显示消息框。
- 注意：方法名开头字母小写，这点与类名的要求不同。
- showMessageDialog 是JOptionPane类中定义的一个“静态（static）”方法
- 静态方法用“类名.方法名（参数列表）”的形式调用，无需事先创建对象。

提前学习



1 什么是静态方法？

无须创建对象即可使用的方法。

2 静态方法用在哪里？

主要用于封装公用的代码。



思索：

为什么java规定作为程序入口点的main()方法是静态的？

结束应用程序

```
System.exit( 0 );
```

- 调用类 System 的静态方法 exit 结束程序
- 参数 0 表明程序顺利结束，非0表示有错误发生



类System 属于包java.lang，这是一个核心包，自动被每个Java程序所引入，不需要import 语句就可以直接使用。

示例三：JavaAppArguments.java

- main()方法的参数是一个字符串数组，保存了从命令行接收到的参数，以下代码提取出所有输入的参数并打印出它们的具体值：

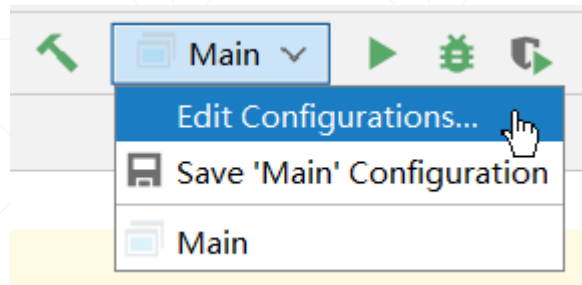
```
public class JavaAppArguments {  
    public static void main(String[] args) {  
        System.out.println("参数个数: " + args.length);  
        //使用foreach循环打印出所有接收到的运行时参数  
        for (String arg : args) {  
            System.out.println(arg);  
        }  
    }  
}
```

命令行参数是在使用java运行Java程序时动态地设定的。

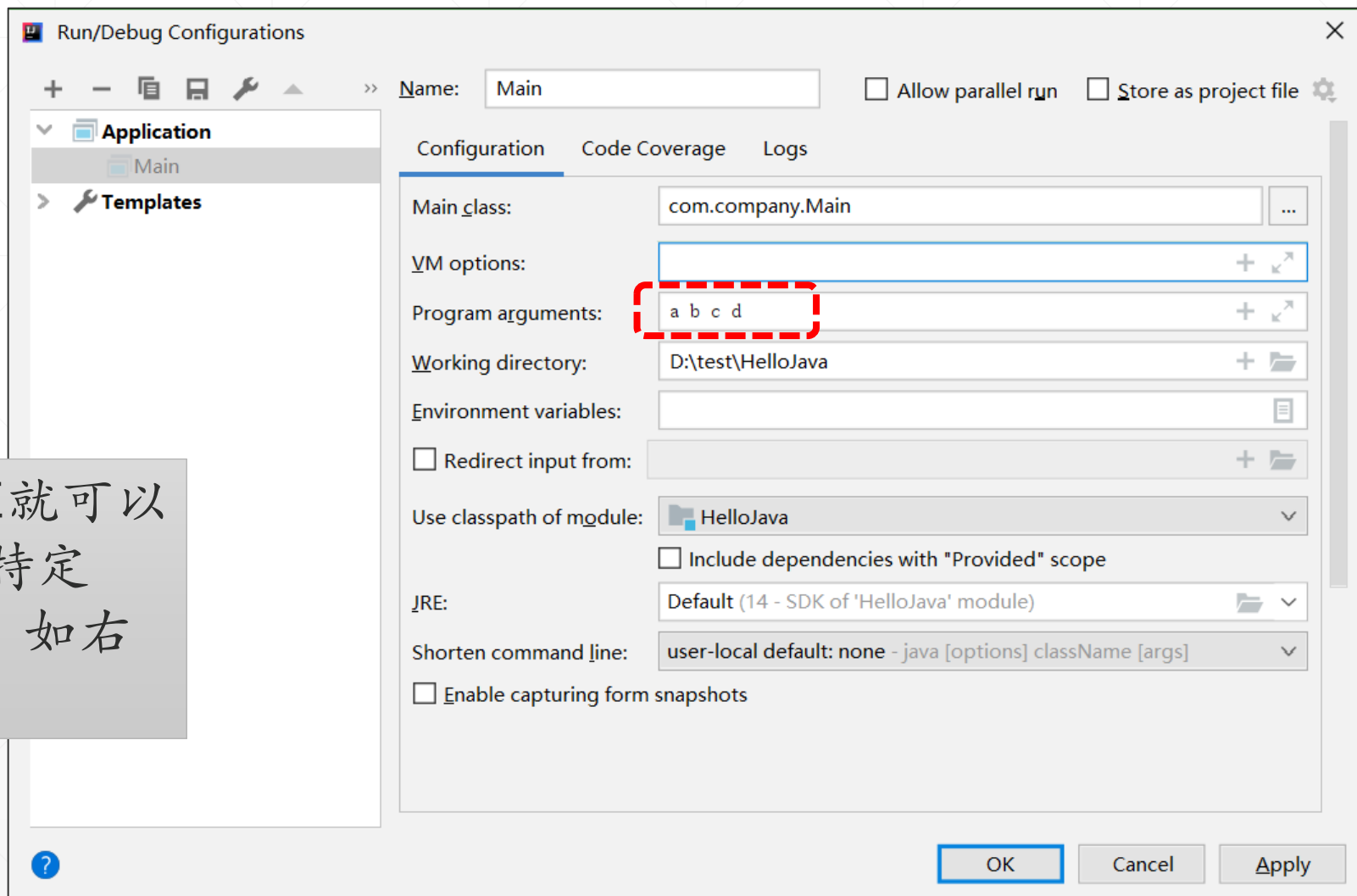


```
Windows PowerShell
PS D:\> javac .\JavaAppArguments.java
PS D:\> java JavaAppArguments a b c d
参数个数: 4
a
b
c
d
PS D:\> java JavaAppArguments "Hello,World!" "这是一个测试"
参数个数: 2
Hello,World!
这是一个测试
PS D:\> |
```

在IntelliJ中指定命令行参数



如果希望能够不离开IDE就可以测试程序，则可以设置特定Java程序的运行时参数，如右图所示。



巩固练习



- 模仿JavaAppArguments.java示例，编写一个程序，此程序从命令行接收多个数字，求和之后输出结果。

提示：

你需要解决以下技术难点：

命令行参数都是字符串，必须先将其转化为数字，才能相加。
如果你不知道如何完成这一工作，请利用互联网解决之。

小结



- 本讲介绍了一种最简单也最基本的Java程序类型，请务必掌握如何使用IntelliJ来编写这种类型的Java程序的基本技巧。
- 这是我们学习的起点，后面的例子大多属于这种程序类型。